

Platform Technical Architecture

Internal Tooling Ecosystem — RMS · EMS · CSAT · CRM · Employee Feedback

Version 1.0 · Confidential · February 2026

1. Platform Overview

devx AI Labs (150+ employees) is building a unified internal platform across four distinct tools, all sharing a single backend, database, and authentication layer. Each tool is a set of Next.js pages within the same monorepo, operating on its own database namespace.



RMS

Resource Management System — allocations, PnL, sprint planning



EMS

Employee Management System — skills, salaries, org chart



CSAT

Client Satisfaction — surveys, NPS, project health scores



CRM Views

Zoho CRM mirror — deal pipeline, account visibility

2. Authentication — Google SSO

SHARED ACROSS ALL TOOLS



Single Sign-On via Google Workspace (@devxailabs.com)

One NextAuth.js session covers RMS, EMS, CSAT, CRM Views, and Employee Feedback. The session token carries `user.id`, `system_role`, and `pod_id`. RBAC permissions are resolved server-side on every API route.



Provider

NextAuth.js v5 with the Google OAuth2 provider. Domain restricted to `@devxailabs.com`. The `hd` (hosted domain) parameter blocks external Gmail accounts.



Session Storage

JWT strategy. Session verified on every server-side API route via `requireSession()`. No cookie store dependency — stateless and horizontally scalable.



RBAC Roles



Auto-provisioning

ADMIN (CEO/CTO/CFO) · **POD_LEAD** (team leads) · **CSM** (client success managers) · **EMPLOYEE** (view-only own data)

New Google Workspace accounts are automatically provisioned as `rms_employees` records via n8n Google Admin webhook. No manual onboarding required.

```
Browser (Chrome)
  | Google OAuth redirect → accounts.google.com
  |       ↓ authorization_code
  ↓
NextAuth.js Callback (/api/auth/callback/google)
  | Verify hd=devxailabs.com
  | Lookup/create rms_employees record
  | Attach system_role + pod_id to JWT
  ↓
Signed JWT (stored in httpOnly cookie)
  |
  |→ Next.js Server Components: getSession()
  |→ API Routes: requireSession() → throws 401 if invalid
  |→ Client Components: useSession() via SessionProvider
```

3. Technology Stack

LAYER	TECHNOLOGY	PURPOSE
Monorepo	Turborepo + npm workspaces	Shared schema, RBAC, types across all tools
Frontend	Next.js 14+ (App Router, TypeScript)	SSR pages, API routes, server components
UI	Tailwind CSS + shadcn/ui	Design system, accessible components
State	TanStack Query + Zustand	Server cache (queries) + local UI state (Gantt drag/copy)
Charts	Recharts	PnL trends, allocation charts, dashboard KPIs
DnD	dnd-kit	Drag-and-drop allocation Gantt
Database	PostgreSQL (AWS RDS Multi-AZ)	Single DB, multiple namespace prefixes
ORM	Prisma v6	Type-safe DB access, migrations, seeding
Auth	NextAuth.js (Google OAuth2)	Workspace SSO, JWT sessions, RBAC
Cache	Redis (ElastiCache)	Session store, API cache (sprint data)
Webhooks	n8n	Zoho CRM sync, Google Admin auto-provisioning
Storage	AWS S3	SoW documents, file attachments

Hosting	AWS ECS Fargate	Container auto-scaling, zero-downtime deploys
CI/CD	GitHub Actions → ECR → ECS	Automated test, build, deploy pipeline
Monitoring	CloudWatch	Logs, alarms, metric dashboards

4. Database Architecture — One DB, Multiple Namespaces

All tools share a single PostgreSQL database. Tables are namespaced by prefix to allow clean joins across tools (e.g., `rms_employees` is the canonical employee source shared by all).

rms_

employees, pods,
projects, accounts,
allocations, sprints,
audit_logs,
pnl_snapshots,
compute_costs

ems_

salary_history, leaves,
promotions,
performance_reviews,
documents, skills
(extended)

csat_

surveys, responses,
nps_scores,
health_checks,
action_items

crm_

zoho_deals, contacts,
activities,
deal_sync_log (mirror
cache of Zoho)

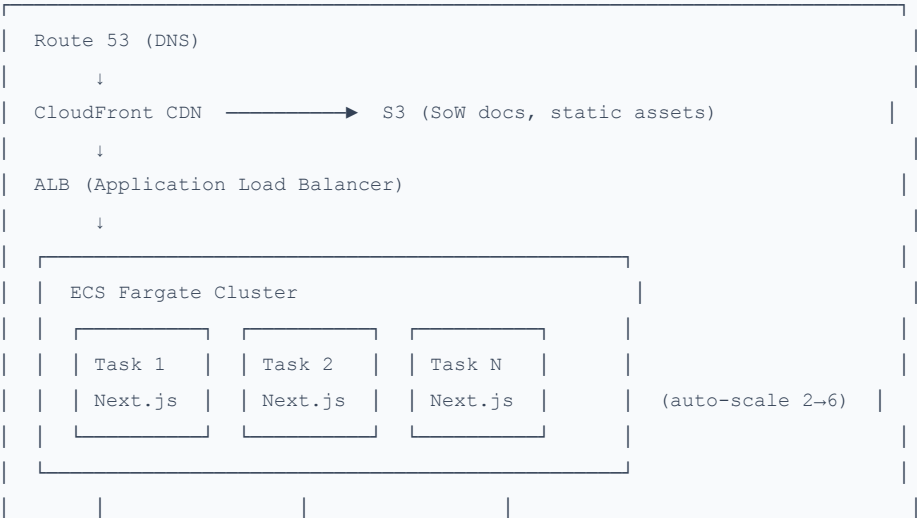
- Key design decisions:**
- All monetary values stored as **integer cents** — no floating point errors
 - All PKs use **UUID** — no enumeration risk
 - All tables carry `created_at` , `updated_at` , `deleted_at` — soft deletes everywhere
 - `rms_employees` is the **canonical employee record** used across RMS, EMS, CSAT, and Feedback tools

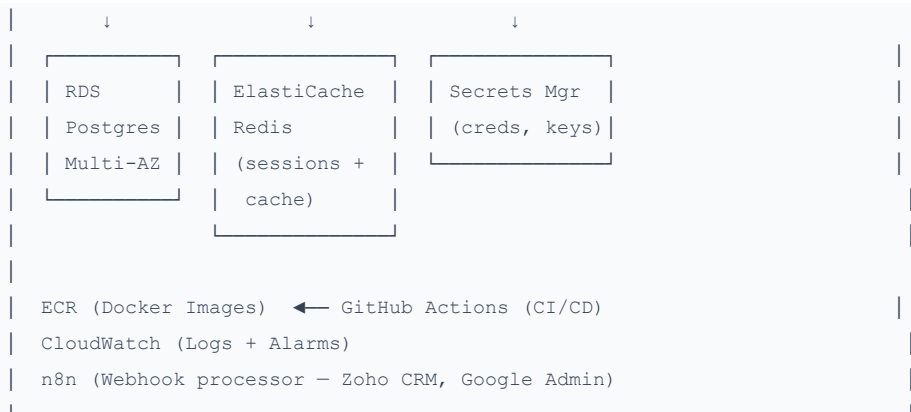
5. Repository Structure

```
devx-platform/                                (Turborepo monorepo)
├─ apps/
│  └─ web/                                    Next.js 14 App Router
│     └─ app/
│        ├── (auth)/login/                   Google SSO login page
│        ├── (dashboard)/                    Protected app shell
│        ├── layout.tsx                       Sidebar + topbar
│        ├── page.tsx                        Dashboard overview
│        ├── employees/                      RMS: Employee directory
│        ├── projects/                      RMS: Project management
│        ├── allocations/                   RMS: Gantt board
│        ├── pnl/                          RMS: P&L overview
│        ├── ems/                          [FUTURE] EMS pages
│        ├── csat/                         [FUTURE] CSAT pages
│        ├── └─ crm/                        [FUTURE] CRM views
│        └─ api/v1/
│           └─ auth/                        NextAuth handlers
```

			employees/	Employee CRUD + activity
			projects/	Project CRUD
			allocations/	Allocations + bulk + history
			sprints/	Sprint management
			pnl/	PnL summary + snapshots
			dashboard/	KPIs + trend data
			webhooks/	n8n: CRM sync, Google Admin
			components/	
			layout/	Sidebar, Topbar
			employees/	EmployeesClient, ProfileDrawer
			allocations/	AllocationGantt, Modals
			projects/	ProjectsClient, DetailDrawer
			pnl/	PnLClient
			lib/	
			auth.ts	NextAuth config
			prisma.ts	Prisma singleton
			services/	Business logic layer
			packages/	
			db/	Prisma schema + migrations
			prisma/schema.prisma	Canonical schema (all namespaces)
			src/	
			seed.ts	Base seed (functions, roles, sprints)
			seed-demo.ts	Demo data (employees, projects, allocs)
			auth/	RBAC permissions map
			types/	Shared TypeScript types
			config/	Env schema (Zod), constants
			turbo.json	
			docker-compose.yml	Local dev: Postgres + Redis

6. AWS Infrastructure





RDS PostgreSQL

Multi-AZ deployment. Automated backups (7-day retention). One database, namespaced tables for all tools. Connection pooling via Prisma connection pool.

ElastiCache Redis

Session token cache. Sprint data caching. TanStack Query also caches aggressively on the client, reducing server round-trips.

ECS Fargate

Serverless containers — no EC2 management. Auto-scales from 2 to 6 tasks based on CPU/memory. Rolling deploys from GitHub Actions pipeline.

7. Data Flow & Webhooks

```

Zoho CRM (Deal Closed Won)
|
↓ HTTP POST (n8n workflow)
n8n Webhook Processor
| Validates deal data + maps fields
| X-Webhook-Secret header auth
↓
POST /api/v1/webhooks/crm
| Creates rms_accounts + rms_projects
| Writes rms_webhook_logs
↓
RMS Platform (project visible immediately)

```

```

Google Workspace (New Employee Added)
|
↓ HTTP POST (n8n workflow)
POST /api/v1/webhooks/google-admin
| Creates rms_employees (status: ACTIVE)
| Sets function_id, role_id from payload
| Logs to rms_webhook_logs

```

↓
Employee can SSO immediately on first login

8. RMS — Resource Management System

PHASE 1–5 COMPLETE

Core Modules

 **Allocation Gantt**

Interactive sprint board. Drag-and-drop (dnd-kit). Per-sprint allocation percentages. 100% cap validation. Bulk copy across sprint ranges. Audit trail per cell.

 **PnL Engine**

Per-project profitability. Revenue vs. employee cost per sprint. Gross margin %. Red-flag detection. Revenue gated behind ADMIN role. Cached snapshots.

 **Sprint Engine**

2-week bi-weekly sprints. Auto-generated for full year. Allocations keyed to `sprint_id`, not date ranges — supports per-sprint % changes.

 **Audit Logs**

Service-layer writes on every allocation create/update/delete. Stores full JSON snapshots (old + new state). Employee activity drawer shows history per person.

 **Pods**

Team groupings with a designated lead. `POD_LEAD` role scopes Gantt read/write access to their pod members only. Editable by ADMIN.

 **Dashboard**

KPIs: active projects, avg allocation (last 6 sprints), available employees (>20% bandwidth), projects in red. Trend chart. Clickable cards navigate to filtered views.

RBAC Matrix

PERMISSION	ADMIN	POD_LEAD	CSM	EMPLOYEE
View all allocations	✓	Pod only	✓	Own only
Edit past sprint allocations	✓	✗	✗	✗
Edit current/future sprints	✓	✓ Pod	✓	✗
View project revenue / margin	✓	✗	✗	✗
Create/edit projects	✓	✗	✗	✗
Create/edit employees	✓	✗	✗	✗
View audit logs	✓	✓	✗	✗

9. EMS — Employee Management System

FUTURE PHASE

EMS extends `rms_employees` as the canonical employee record. All EMS tables use the `ems_` prefix and foreign-key into `rms_employees.id`.

Key Tables

ems_salary_history	(employee_id, effective_date, ctc_cents, reason, approved_by)
ems_leaves	(employee_id, leave_type, from_date, to_date, status)
ems_promotions	(employee_id, old_role_id, new_role_id, effective_date, notes)
ems_performance_reviews	(employee_id, period, rating, reviewer_id, summary)
ems_documents	(employee_id, type [offer/appraisal/nda], s3_key, uploaded_at)
ems_skills	(employee_id, skill_id, level [1-5]) ← extends rms_skills

 Salary & Payroll

Salary history with effective dates. CTC stored as cents (integer). Syncs to `rms_employees.salary_ctc_cents` for PnL cost calculations.

 Skills & Matching

Skill proficiency (1–5 levels).
Searchable via `GET /api/v1/ems/employees?skill=AI&min_level=3`.
Enables smart resource allocation suggestions.

 Document Vault

Signed URLs to S3 for offer letters, appraisals, NDAs.
Accessible via ADMIN and the employee themselves. Enforced by Prisma + S3 IAM policies.

SSO Note: EMS uses the same Google SSO session. An employee accesses their own EMS profile at `/ems/me`. HR and ADMIN can access all profiles at `/ems/employees`. No separate login — one session, all tools.

10. CSAT — Client Satisfaction FUTURE PHASE

 Survey Delivery

NPS / CSAT surveys triggered by sprint completion or project milestones (via n8n). Delivered to client contacts via email. Responses stored in `csat_responses`.

 Project Health Scores

Computed score per project per sprint from survey responses. Displayed on the RMS project detail drawer. Alerts on health score drop below threshold.

Key Tables

csat_surveys	(project_id, trigger_type [sprint_end/milestone], sent_at, status)
csat_responses	(survey_id, respondent_email, nps_score, rating, comment, submitted_at)
csat_health_checks	(project_id, sprint_id, computed_score, is_at_risk)
csat_action_items	(health_check_id, description, owner_id, due_date, status)

SSO Note: Internal CSAT dashboards (`/csat`) use the shared Google SSO session. External client respondents receive a public survey link — no SSO required. Response collection is anonymous by default.

11. CRM Views — Zoho Mirror

FUTURE PHASE

Rather than rebuilding a CRM, devx reads Zoho CRM data via n8n webhooks + scheduled syncs, stores a local cache in `crm_` tables, and surfaces read-only views inside the platform.

```

Zoho CRM — n8n Webhook (Closed Won) → POST /api/v1/webhooks/crm
                                     |
                                     Creates rms_projects + rms_accounts
                                     Caches in crm_zoho_deals

Zoho CRM — n8n Scheduled Sync (nightly) → POST /api/v1/webhooks/crm/sync
                                     |
                                     Updates deal stages, contact info
                                     Marks stale deals as CANCELLED
  
```

Bi-directional Sync

Webhook for instant Closed Won trigger. Nightly n8n job for deal stage updates (e.g., Active → On Hold if Zoho changes). RMS is the downstream consumer, Zoho is the source of truth for deals.

CRM View Pages

Read-only pipeline view at `/crm`. Shows deal stage, value, contact. Links through to the RMS project when a deal is Closed Won. No write-back to Zoho from RMS.

12. Employee Feedback Ecosystem

FUTURE PHASE

360° Reviews

Peer reviews, upward feedback, and self-assessments per review cycle. Stored in `feedback_reviews`. Aggregated into EMS performance records.

Goal Tracking

OKRs per employee per quarter. `feedback_goals` table. Manager can set, review, and close goals. Linked to performance review cycles.

Recognition

Peer shoutouts and kudos. `feedback_recognition` table. Surfaced on employee profiles. Optional Slack notification via n8n workflow.

SSO Note: All feedback flows through the same Google SSO session. Employees submit peer reviews at `/feedback`. Anonymity is enforced at the API layer — feedback is aggregated before managers see it. HR and ADMIN see individual responses.

13. Implementation Phases



Phase 0 — Foundation

Turborepo scaffold · Prisma schema · NextAuth Google SSO · Docker dev env · Sprint seeding · Full UI shell deployed



Phase 1 — Employees

Employee CRUD · POD management · Google Admin webhook auto-provisioning · Employee directory tab with allocation overview

 Phase 2 — Projects

Accounts + Projects CRUD · Zoho webhook · Projects tab with RBAC revenue gating · Bulk CSV import

 Phase 3 — Allocation Gantt

Sprint allocation engine · 100% cap validation · dnd-kit drag-and-drop · Copy-paste bulk · Allocation history audit drawer

 Phase 4 — PnL

PnL summary API · Per-project cost engine · Gross margin · Red-flag detection · Expandable employee cost breakdown · Admin-gated revenue columns

 Phase 5 — Dashboard

KPI cards · Avg allocation across last 6 sprints · Available employees (>20% bandwidth) · Projects in red · Clickable navigation to filtered views

 Phase 6 — EMS Foundation

ems_ tables · Salary history · Skill management · Leave tracking · Employee self-service portal

 Phase 7 — CSAT

Survey delivery · NPS scoring · Project health checks · CSAT dashboard

 Phase 8 — CRM Views + Feedback

Zoho deal pipeline view · 360° reviews · Goals · Recognition system

14. API Surface Summary

DOMAIN	KEY ENDPOINTS	AUTH
Auth	<code>GET /api/auth/session</code> · <code>POST /api/auth/signin</code>	Public
Employees	<code>GET/POST /api/v1/employees</code> · <code>PATCH /api/v1/employees/:id</code> · <code>GET /api/v1/employees/:id/activity</code>	ADMIN write; all read
Projects	<code>GET/POST /api/v1/projects</code> · <code>GET /api/v1/projects/:id</code>	ADMIN write; revenue gated
Sprints	<code>GET /api/v1/sprints</code> · <code>GET /api/v1/sprints/current</code>	All
Allocations	<code>GET /api/v1/allocations/gantt</code> · <code>POST /api/v1/allocations/bulk</code> · <code>POST /api/v1/allocations/copy</code>	Role-gated by sprint/pod
PnL	<code>GET /api/v1/pnl/summary</code> · <code>GET /api/v1/pnl/:projectId</code>	Revenue: ADMIN only
Dashboard	<code>GET /api/v1/dashboard</code>	All (filtered by role)

Webhooks	<code>POST /api/v1/webhooks/crm</code> · <code>POST /api/v1/webhooks/google-admin</code>	X-Webhook-Secret header
----------	--	-------------------------